

G.H. Raisoni College of Engineering and Management

Department of Mechanical Engineering

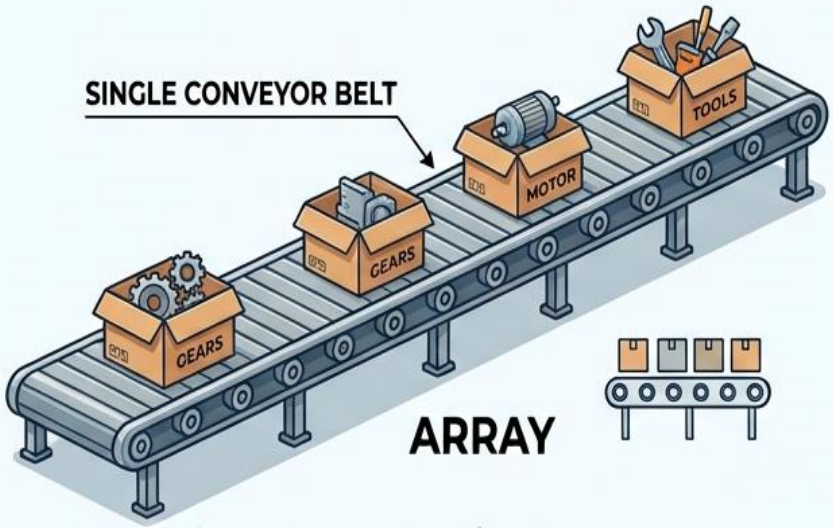
Unit 5 :Trees , Graphs and Hashing

Prof. Sagar Mahajan

THE MECHANICAL ANALOGY: WHY TREES?

Visualizing Data Structures with Mechanical Concepts

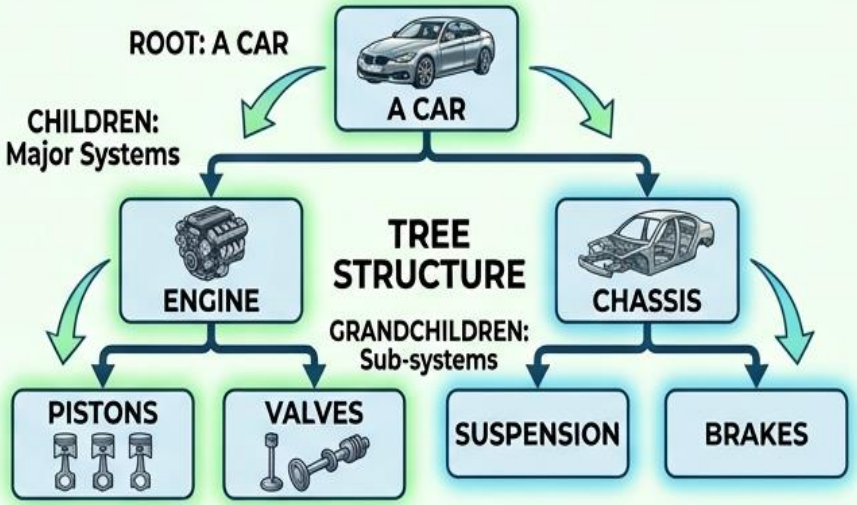
ARRAY (LINEAR)



One item after another. **Single sequence.**

Analogy: Simple sequential assembly.

TREE (NON-LINEAR)



To represent **hierarchical relationships.** Deep nested structures.

Analogy: Bill of Materials (BOM) & Company Hierarchy.
This tree model fits complex objects like a car.

EXPLANATION

Analogy: A single continuous line. One item is placed on the belt, and another follows directly behind it in a simple linear order. This is a sequence of items, with no branching, representing how an array stores data elements one after another in contiguous memory.

Analogy: More complex objects, like cars or organizations, are not simple flat lists. They are best represented by hierarchical structures, similar to a Bill of Materials (BOM) or a corporate hierarchy. The overall system is broken down into major components, which are further divided into sub-components, and so on. This hierarchical tree structure elegantly models these deep nested relationships, allowing for efficient representation, navigation, and searching within structured systems.

BASIC TERMINOLOGY: THE FAMILY TREE

ROOT:

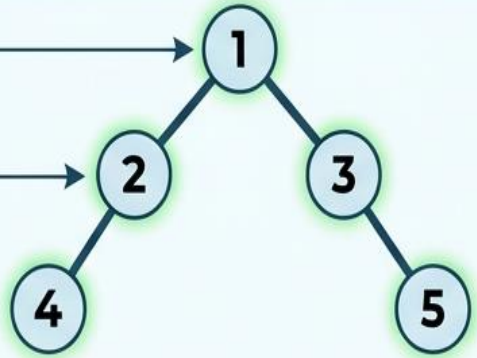


MAIN CAR ASSEMBLY
The topmost node.
(The main car assembly).

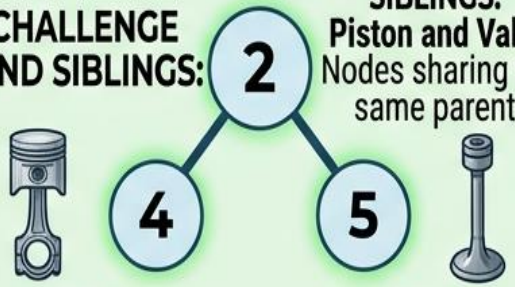
PARENT:



ENGINE
A node that has branches below it.
(The Engine).



CHALLENGE AND SIBLINGS:

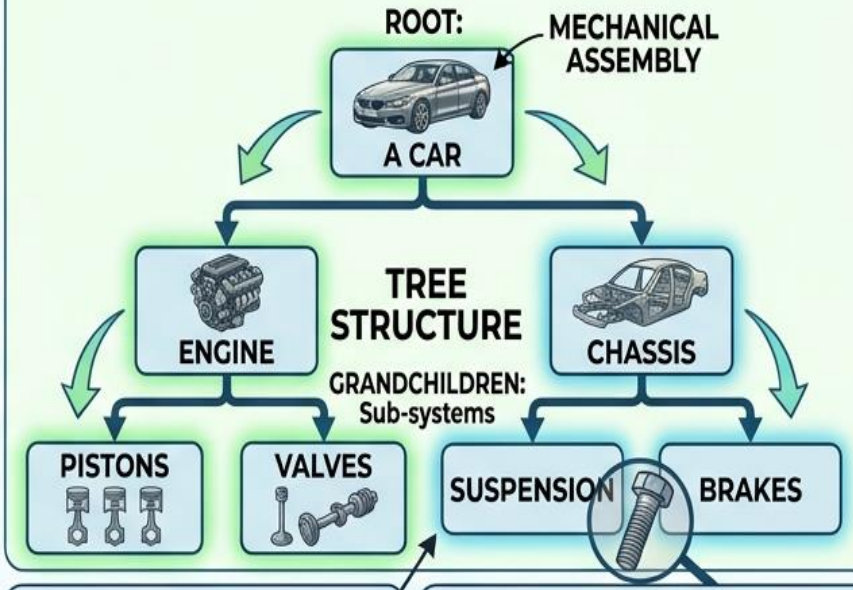


SIBLINGS:
Piston and Valve
Nodes sharing the same parent.

CHILD: Piston
A node directly connected below a parent. (The Piston).

CHILD: Valve
A node directly connected below a parent. (The Valve).

Mechanical Family Tree





LEAF:
A node with no children.
(The Spark Plug).



EDGE:
The connecting link between two nodes.
(The bolts connecting them).

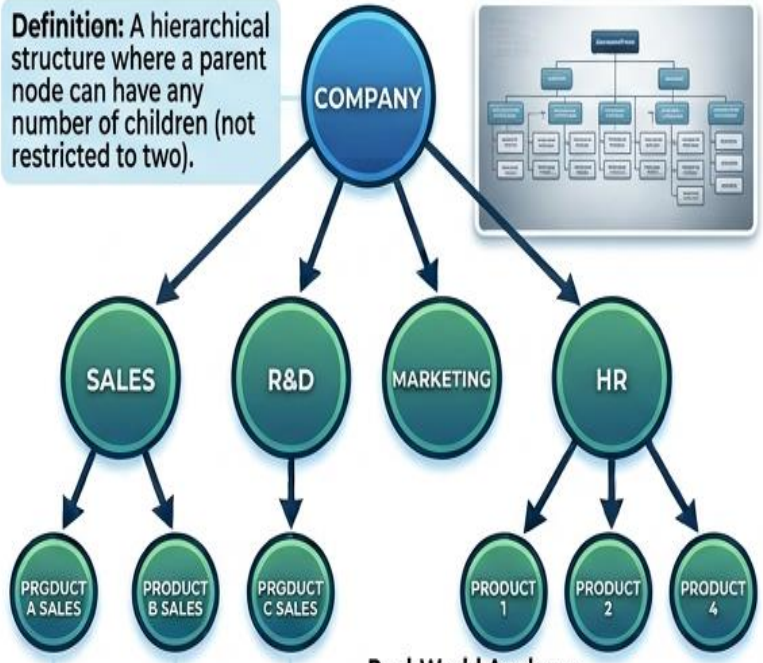
EXPLANATION:
In programming, we call these 'Nodes'. Imagine a mechanical assembly. The top part is the Root. It connects to Parents. The Parents connect to Children. The very end parts that have nothing below them are called Leaves. This is the basic vocabulary.

This hierarchy elegantly models structured systems.

Types of Trees (Binary & BST)

1. General Tree

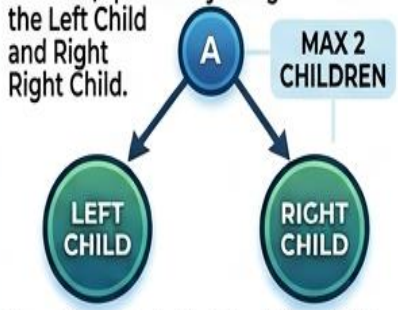
Definition: A hierarchical structure where a parent node can have any number of children (not restricted to two).



Real-World Analogy:
A nalogy A company organizational chart (e.g., Company → Departments → Teams)

2. Binary Tree

Definition: A specialized tree where every parent node can have at most 2 children, specifically designated as the Left Child and Right Right Child.



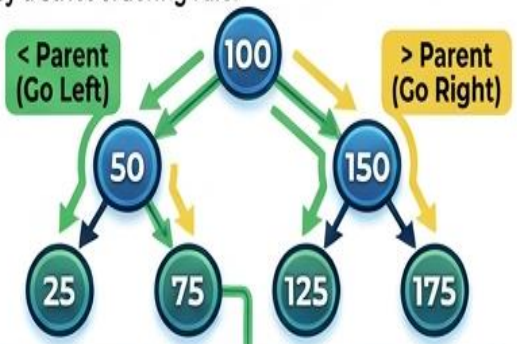
Key Characteristic: Simplifies decision-making paths (like a binary switch.



Simple Binary Tree (Max 2 Children) - used in Expression Trees, Game States

3. Binary Search Tree (BST)

Definition: A special type of Binary Tree governed by a strict ordering rule:



Real-World Analogy: A well-organized tool crib. Smaller tools and parts are placed on the left shelves.



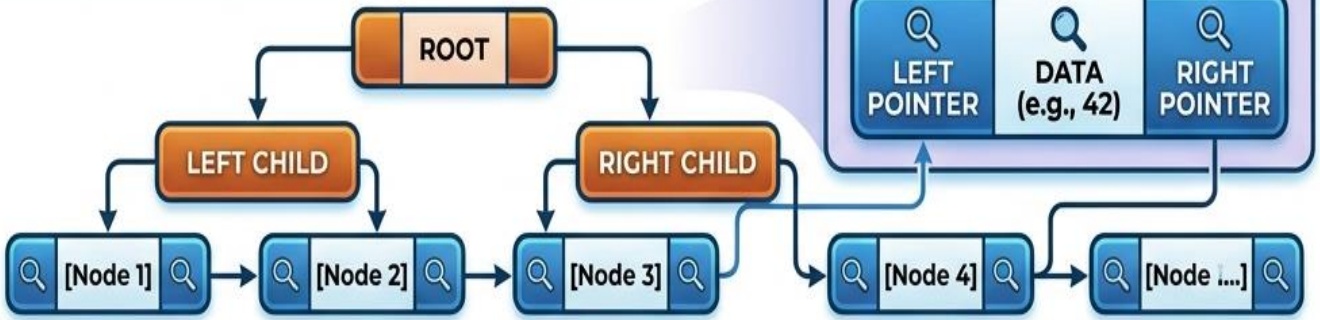
Explanation & Key Characterics

Core Concept: While a General Tree handles arbitrary hierarchical relationships, a Binary Search Tree (BST) introduces sorted order. This ordered structure **eliminates linear scanning**, transforming search operations into efficient logarithmic navigation.

Feature	General Tree	Binary Search Tree (BST)
Child Limit	Arbitrary (Any number of children)	Maximum of 2 children (Left and Right)
Ordering Rule	None (Unordered hierarchy)	Left < Parent < Right
Search Efficiency	Requires traversal of all branches	Logarithmic search time (O(log n))

REPRESENTATION 1 - LINKED LIST

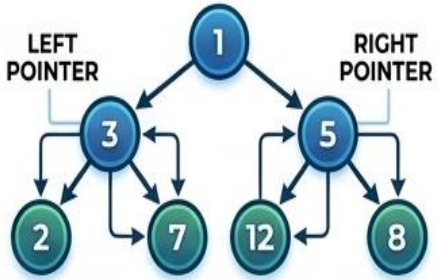
VISUAL REPRESENTATION



Node Box Structure



TREE STRUCTURE FROM LINKED NODES

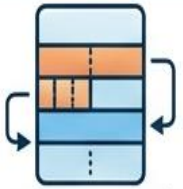


CONTENT

HOW IT WORKS:



Each node is created **dynamically** using **malloc()**.



Memory Allocation

STRUCTURE:

```
struct Node {
    int data;          ——— data,
    struct Node* left;
    struct Node* right;
};
```

PROS:



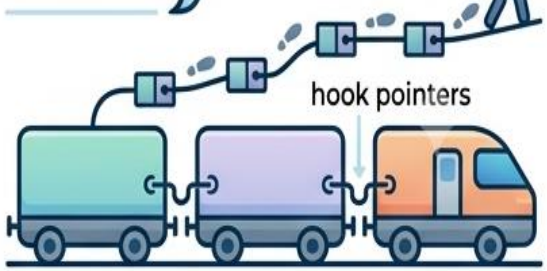
Dynamic size. Add a new part without rebuilding the whole system. delete a node.

CONS:



Extra memory needed to store the pointers.

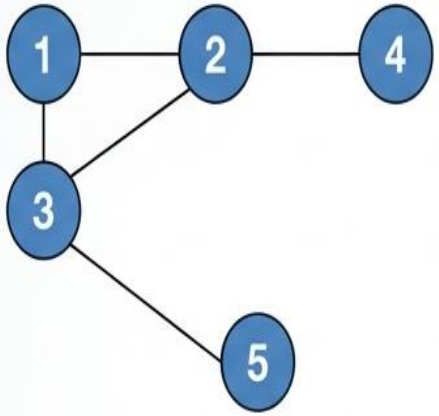
No direct access. You must follow the links.





EXPLANATION: *This is the most common way to build a tree in C. It's like a train where each car has two hooks. One hook connects to the left car, one to the right. You can add or remove cars easily without breaking the whole train. But every car needs extra space for those hooks (pointers).*

REPRESENTATION 2 - ADJACENCY MATRIX / ARRAY



	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	0
3	1	0	0	0	1
4	0	1	0	0	0
5	0	0	1	0	0

HOW IT WORKS:

A 2D array (Matrix) of size N x N, where N is the number of nodes.

RULE:

If there is a connection between Node A and Node B, put a 1. Otherwise, put a a 0.

PROS:

Very fast to check if two nodes are connected.

CONS:

Wastes memory. If you have 100 nodes but only 10 connections, you still need a 100x100 grid (lots of zeros).

EXPLANATION

	A	B	C	D	E	F	G	H	I
1 Part A	0	1	1	0	0	0	0	0	0
2 Part B	1	0	0	1	0	0	0	0	0
3 Part C	1	0	0	0	1	0	0	0	0
4 Part A	0	1	0	0	0	0	0	0	0
5 Part 5	0	0	1	0	0	0	0	0	0

Imagine a massive Excel sheet. The rows are all the parts of your machine, and the columns are also all the parts. If Part A connects to Part B, you put a 1 in that box. This is Adjacency Matrix.
It is great for computers to read quickly, but it wastes a lot of memory if the tree is sparse.

TREE TRAVERSALS

(HOW TO WALK THE TREE)

VISUAL REPRESENTATION & TRAVERSAL FLOW

1. PREORDER TRAVERSAL

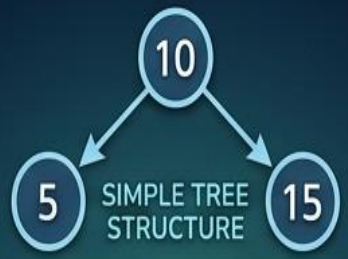


2. INORDER TRAVERSAL



3. POSTORDER TRAVERSAL





CONTENT

1. PREORDER
(Root -> Left -> Right)
Visit the Parent first.
(10, 5, 15)

2. INORDER
(Left -> Root -> Right)
Visit the Left child, then Parent, then Right.
(5, 10, 15)
Note: This gives sorted data in a BST!

3. POSTORDER
(Left -> Right -> Root)
Visit the children first, Parent last.
(5, 15, 10)

MECHANICAL ANALOGY & EXPLANATION

MECHANICAL ANALOGY



Designing the car.



Assembling the car.



Testing the car.

EXPLANATION

Traversal just means "how do we visit every node?" In a linear array, it's easy: 1, 2, 3. In a tree, we have choices.



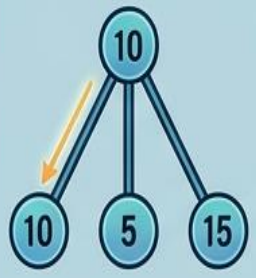
Preorder means the boss speaks first, then the workers. **Postorder** means the workers finish their job, then then the boss speaks. **Inorder** is special because it outputs the data in perfect sorted order.

THE MECHANICAL ANALOGY (WHAT IS A GRAPH?)

Visualizing complex network data structures using mechanical systems.

1. VISUAL COMPARISON: TREE VS. GRAPH

TREE: A STRICT HIERARCHY



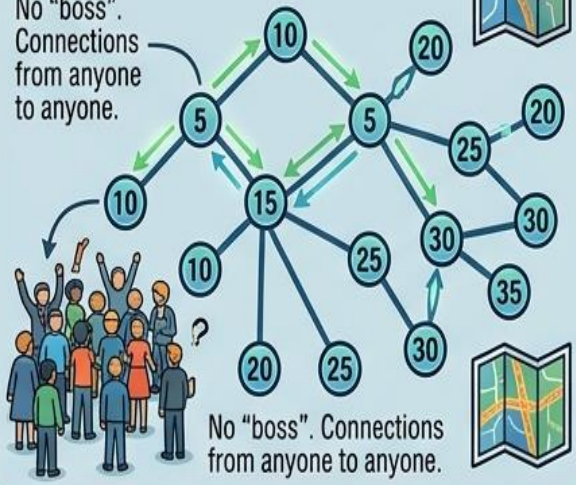
(Boss → Worker). Strict structure.



Note: A **Tree** is just a specialized type of **Graph** (no loops/cycles).

GRAPH: A NETWORK

No "boss". Connections from anyone to anyone.



No "boss". Connections from anyone to anyone.

2. CORE GRAPH CONCEPTS & DEFINITION

WHAT IS A GRAPH?

A collection of **Vertices** (points) connected by **Edges** (lines).



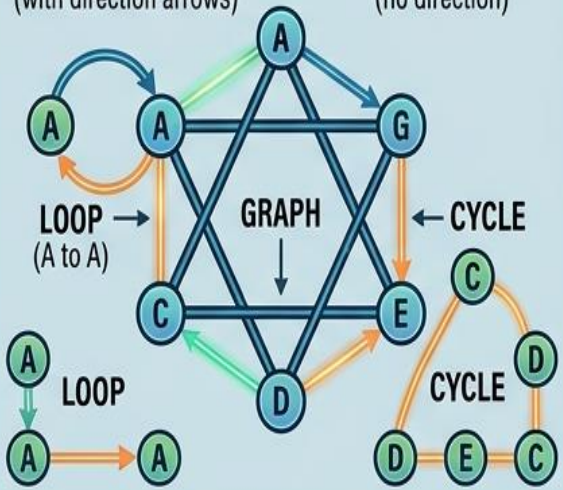
TYPES OF GRAPHS



DIRECTED GRAPH (with direction arrows)

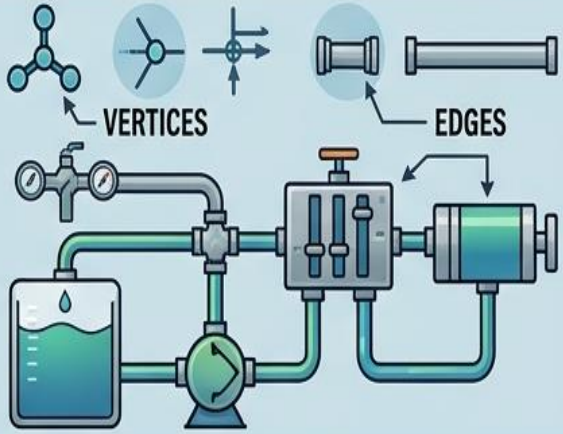


UNDIRECTED GRAPH (no direction)



3. MECHANICAL ANALOGY & EXPLANATION

MECHANICAL ANALOGY: HYDRAULIC CIRCUIT



EXPLANATION

Think of a tree as a company hierarchy. A **graph** is a **network of roads** or a **hydraulic circuit.** In a hydraulic system, oil can flow from pump to **valve**, **valve** to **cylinder**, and **cylinder** back to tank. There's no strict "parent-child" hierarchy; it's a network. That's a **Graph.**

APPLICATION EXAMPLES



SOCIAL NETWORKS



GPS NAVIGATION



POWER GRIDS

CORE TERMINOLOGIES (THE VOCABULARY)

GENERAL GRAPH CONCEPTS

VERTEX (Node)

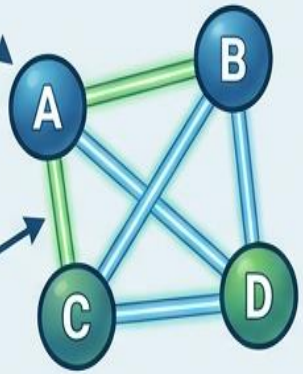


The junction point.
Example: Vertex A, B, C, D

EDGE (Link)

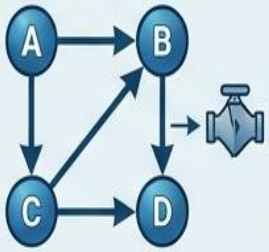


The pipe connecting two vertices.
Example: Edge (A,B), (B,C), etc.



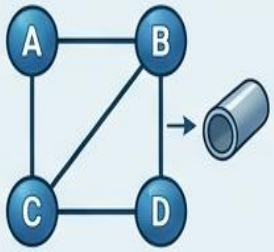
VISUAL 1.1: GENERAL GRAPH ELEMENTS

DIRECTED GRAPH (Digraph)



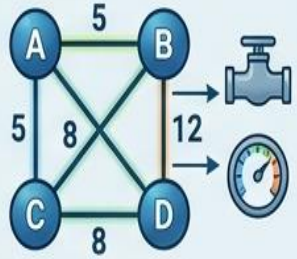
Flow is **one-way** (like a check valve). Edges have arrows.

UNDIRECTED GRAPH



Flow is **two-way** (like a normal pipe). Edges have no arrows.

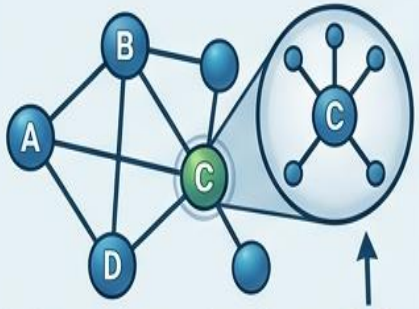
WEIGHTED GRAPH



Edges have a **value** (e.g., pipe length, pressure drop, travel time).

VISUAL 1.2: TYPES & PROPERTIES OF GRAPHS

KEY GRAPH PROPERTIES & STRUCTURE



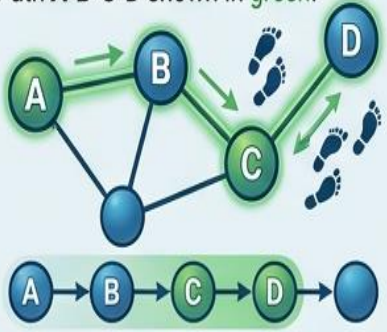
DEGREE: The number of edges connected to a vertex.
(How many pipes meet at this junction?).
For Node C, **Degree = 4.**



Degree

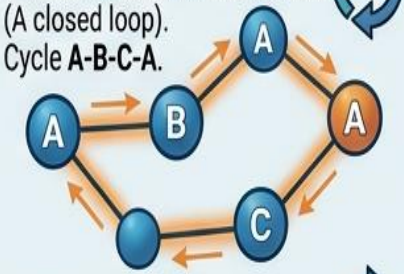
VISUAL 2.1: VERTEX DEGREE & PROPERTIES

PATH: A sequence of vertices connected by edges.
(The route the oil takes).
Path **A-B-C-D** shown in green.



VISUAL 2.2: PATH & STRUCTURE

CYCLE: A path that starts and ends at the same vertex.
(A closed loop).
Cycle **A-B-C-A.**



CYCLE: A path that starts and ends at the the same vertex.
Cycle **A-B-C-A** shown in orange.

VISUAL 2.3: CYCLE & CLOSED LOOPS

EXPLANATION & SUMMARY

EXPLANATION:
Terminology is crucial.

If you have a **one-way valve,** your graph is '**Directed.**'

If it's a normal pipe, it's '**Undirected.**'

If you care about how much **pressure drops** across a pipe, you put a number on the edge—that makes it a '**Weighted Graph.**'

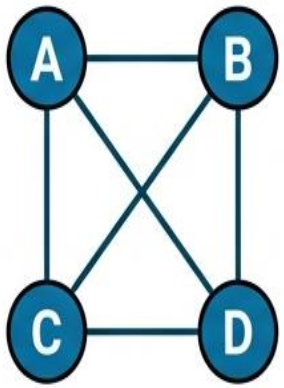
The '**Degree**' is simply how many pipes connect to a single junction.

Representation 1 - Adjacency Matrix

How it works: A 2D Array (Matrix) of size V x V (where V is number of vertices).

Rule: If there is a connection between Vertex A and Vertex put a 1. Otherwise, put a 0. (For Weighted Graphs, put the weight instead of 1).

	A	B	C	D
A	0	1	0	0
B	1	1	0	1
C	0	0	1	0
D	0	1	0	1



Pros:
Extremely fast to check if two junctions are connected (O(1) time).

Cons:
Wastes memory. If you have 1000 junctions but only 5 pipes, you still need a 1000x1000 grid (mostly zeros).

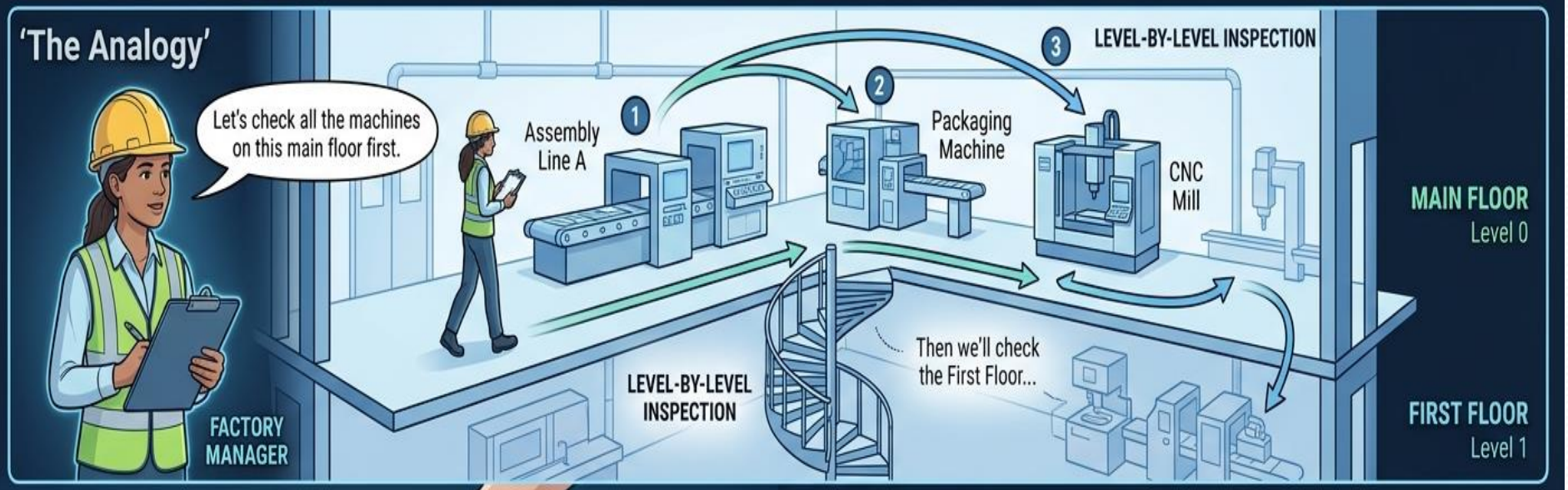
EXPLANATION

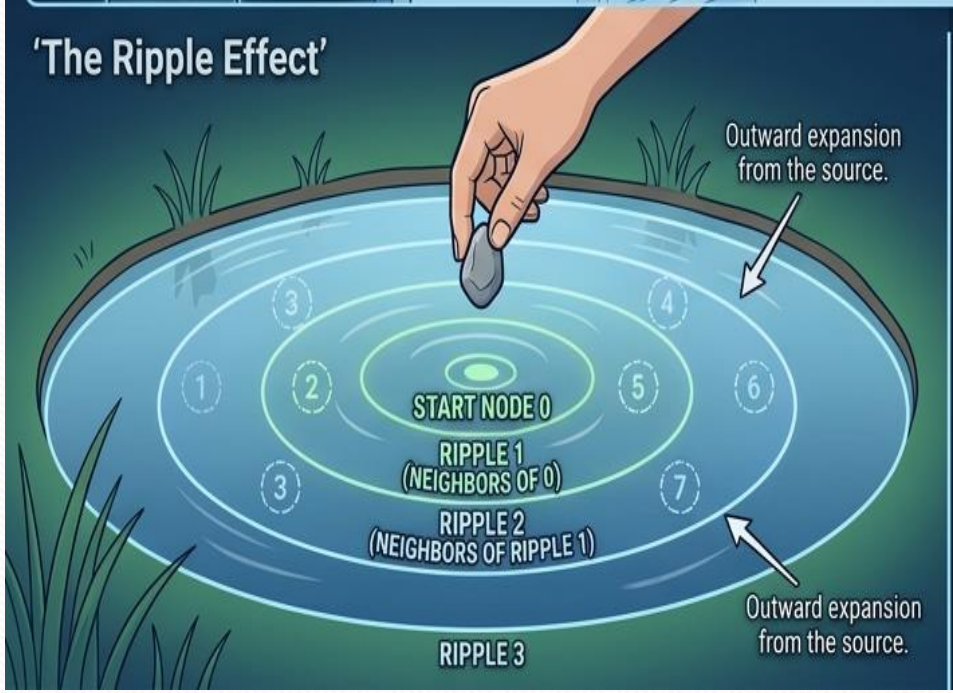
Imagine an Excel sheet. Rows are all the junctions. Columns are all the junctions. If a pipe connects Junction A to Junction B, you put a 1 in that box. This is great for computers to read quickly, but it is like printing a 1000-page phonebook when you ly have 5 phome numbers. It wastes a lot of memory.

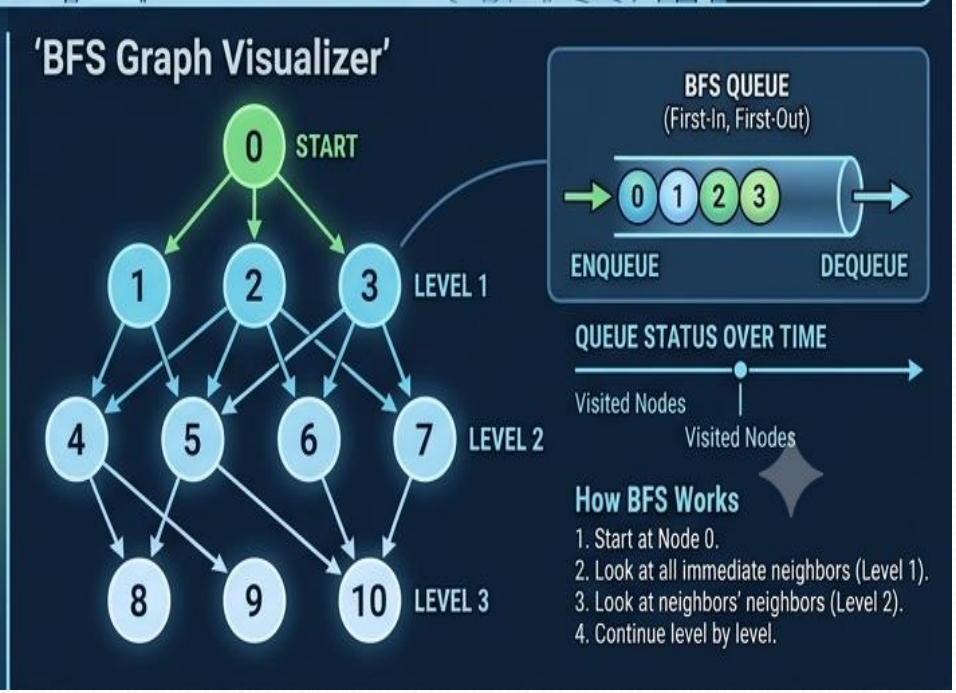
Breadth First Search (BFS)

- Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level.
- First, it visits all nodes directly adjacent to the source.
- Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

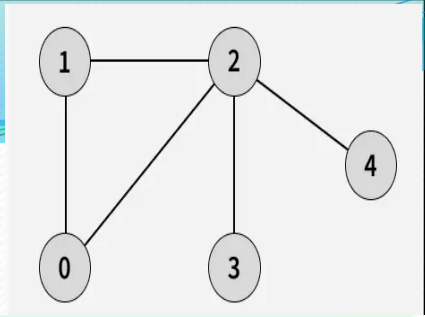
BFS (Breadth-First Search) - 'The Factory Manager'





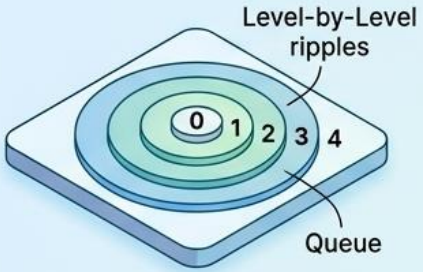


Breadth First Search (BFS) Example



Step-by-Step BFS Traversal Table

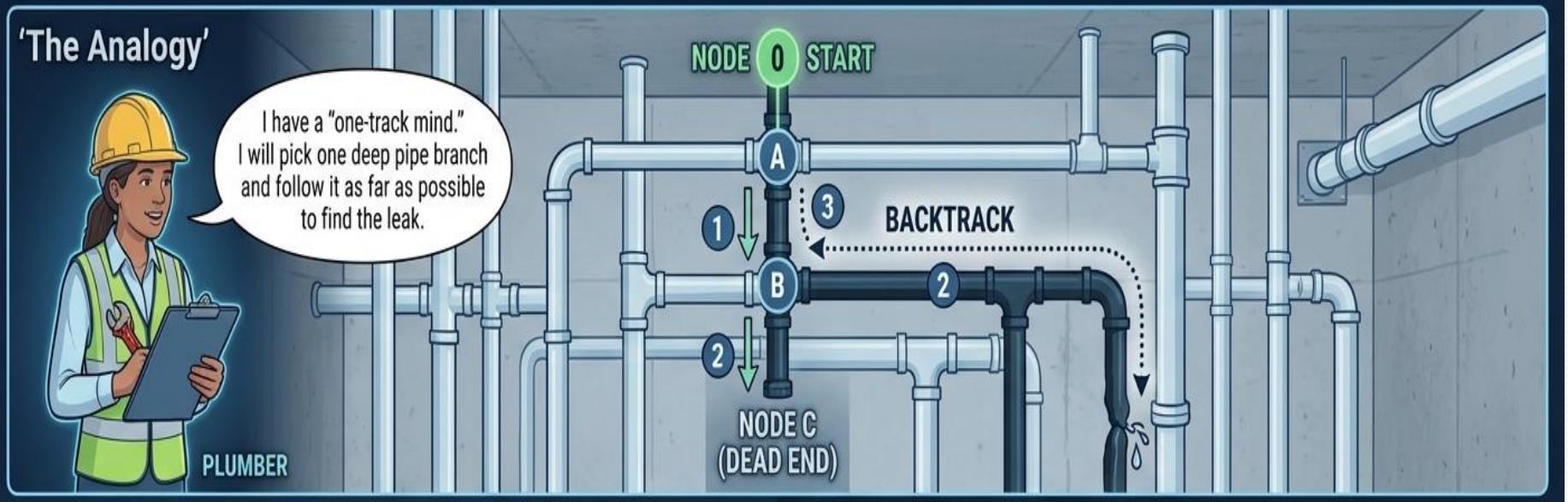
Step	Current Node Processed	Neighbors Inspected	Unvisited Neighbors Enqueued	Queue Content ⇨ (FIFO) ⇨	Visited Set	Traversal Output Order
Start	—	—	—	[0]	{0}	**0**
1	**0**	1, 2	1, 2	[1, 2]	☑ {0, 1, 2}	**0**
2	**1**	0, 2	*None* (both visited)	[2]	☑ {0, 1, 2}	**0, 1**
3	**2**	0, 1, 3, 4	3, 4	[3, 4]	{0, 1, 2, 3, 4}	**0, 1, 2**
4	**3**	2	*None* (visited)	[4]	{0, 1, 2, 3, 4}	**0, 1, 2, 3**
5	**4**	2	*None* (visited)	[ ] *(Empty)*	{0, 1, 2, 3, 4}	**0, 1, 2, 3, 4**

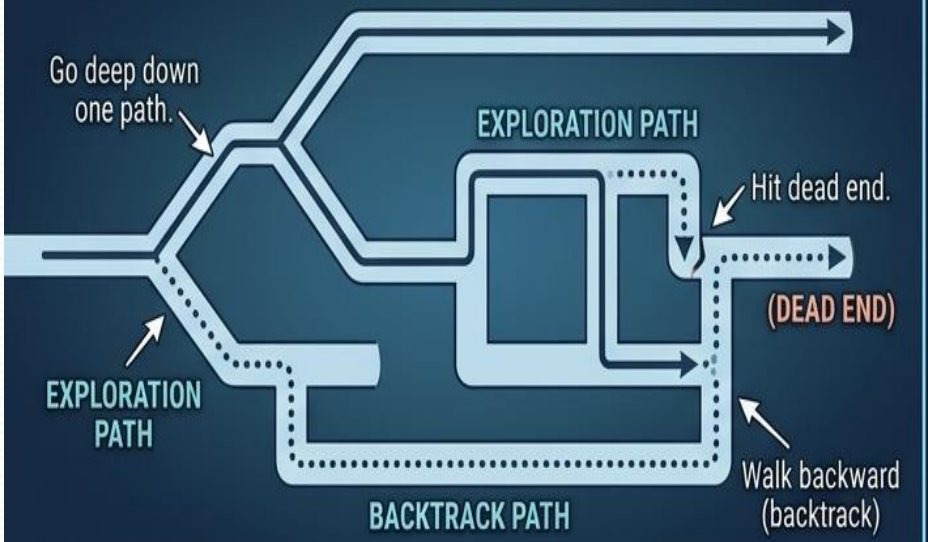


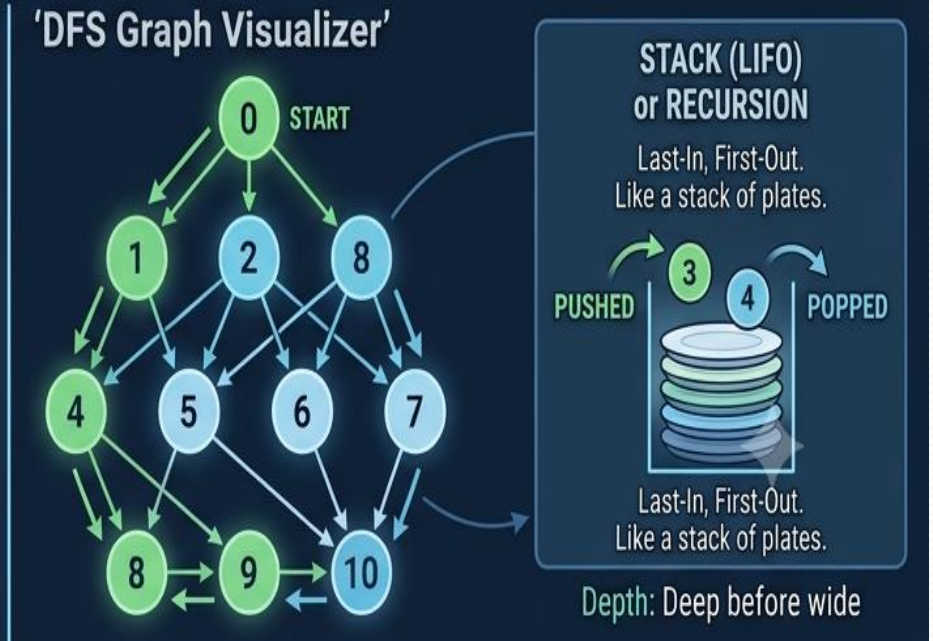
Key Presentation Takeaways for Students:

- **Starting Node: 0**
- BFS uses a **Queue** data structure.
- Traversal order is **level-by-level** (all neighbors first).

2. DFS (Depth-First Search) - "The Plumber"







Hash Tables (The Fast Tool Crib)

Find what you need, instantly !

The Problem

Searching for **1** part in **10,000** parts takes too long.

O(n) time
(in Arrays / Linked Lists)



The Solution

Hash Table

A data structure that stores data in key-value pairs.

Key (e.g. part name) → Value (e.g. drawer number)



How it works

1. Take the key (e.g. "M8 Bolt").
2. Apply a hash function (to calculate the drawer number).

   hash("M8 Bolt") → **4**
3. Go directly to that index and get the value.

No searching. Just calculate and go!

O(1) time
(Constant time)

Mechanical Analogy

A perfectly organized tool crib. If you need an M8 bolt, you don't search every box. You go straight to the box labeled "M8 Bolts."



Example

Part	Drawer (Index)
M8 Bolt	4
Valve	7
Gear	2
Filter	9

Hash Tables = Fast Search

Same data... Just a smarter way to find it!

Hash Functions (The Sorting Rule)

A simple formula with a powerful job!

What is it?

A mathematical formula that takes a **Key** (like a Part ID or Student Roll No.) and converts it into an **Index** (Drawer Number).

Key (Part No. / Roll No.) → **Index** (Drawer No.)

It's like a sorting rule for your data!

The Modulo Formula

Index = Key % Table_Size

Example

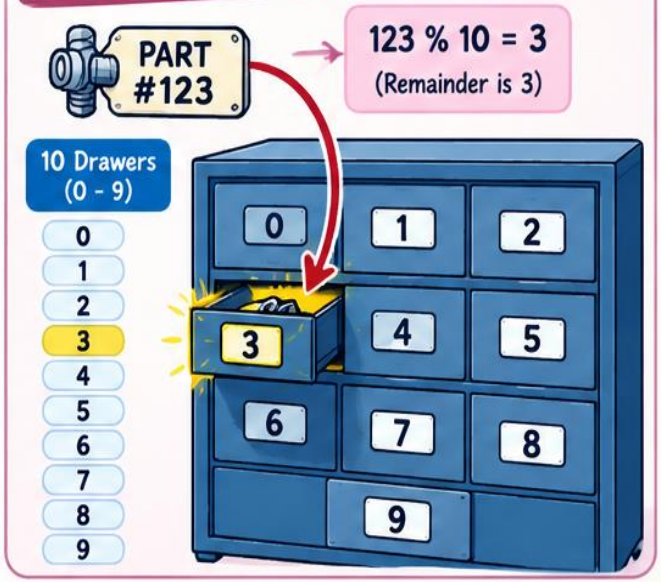
Key = 123

Table Size = 10

Calculation = 123 % 10 = **3**

So, store it in slot 3!

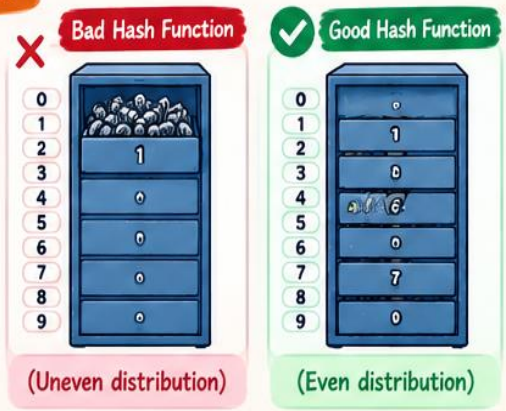
Visual Example



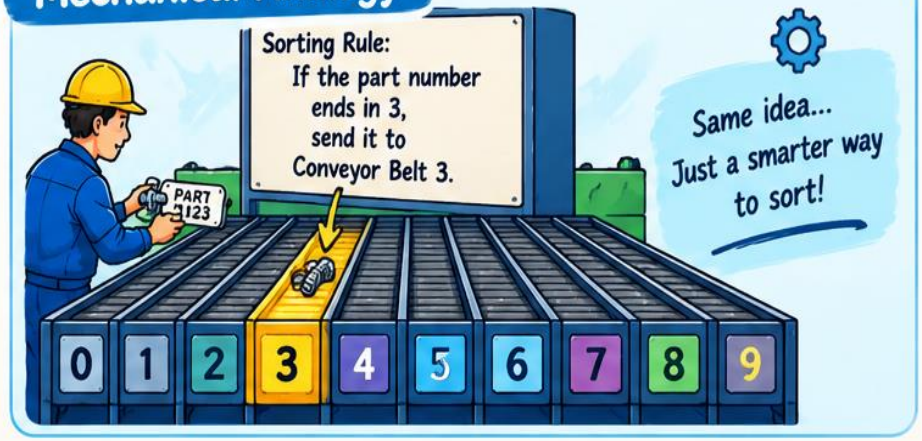
Good Hash Function

Should distribute parts evenly.

You don't want all parts in drawer 1 and empty drawers everywhere else.



Mechanical Analogy



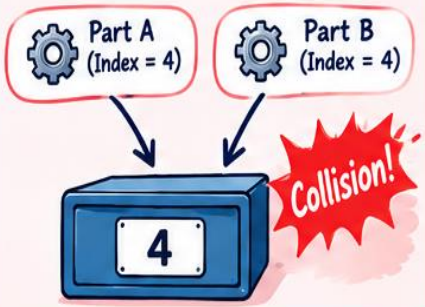
Collision Resolution (Chaining)

Same drawer... multiple items... No problem!

The Collision Problem

What if Part A and Part B both calculate to **Index = 4 ?**

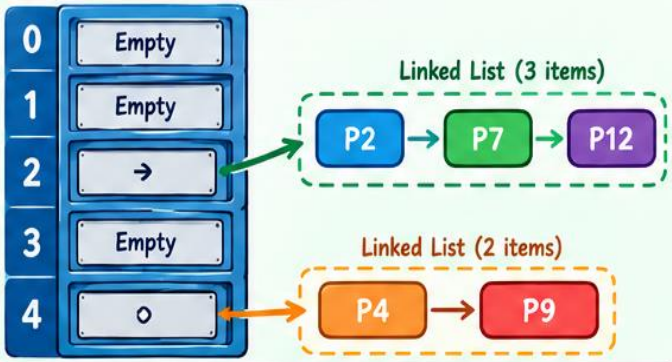
(Two tools need the same drawer).



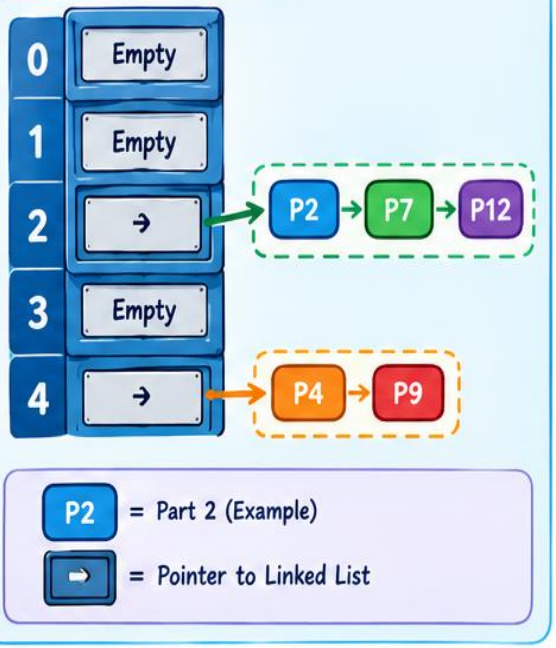
Solution 1: Chaining (Open Hashing)

- Instead of one item per slot, we attach a **Linked List** (a chain) to each slot.
- If two parts need Drawer 4, they both hang on the same chain.

Hash Table (Array of 5 Slots)



Visual Example



Mechanical Analogy



You have a hook for "Size 10 Wrenches."

You have 3 wrenches of size 10.

You don't build a new hook; you just hang all 3 wrenches on the same hook.

Same drawer, multiple items!

Key Takeaways

- **Collisions are normal** – With a limited number of slots, two keys can map to the same index.
- **Chaining solves it** – Use a linked list at each slot.
- **It's simple** – No extra space for empty slots.
- **It keeps searching fast** – You only check the short chain at the index, not the whole table.

Same slot... Just a longer chain!

Collision Resolution (Open Addressing)

No chains.
No linked lists.
Just find the next spot!

The Idea

Solution 2: Open Addressing (Closed Hashing).

- No linked lists.
- Everything stays inside the main array.

Simple. Fast. Memory friendly.

The Parking Lot Example



How it works (Probing)

1. Calculate the index (e.g. 4). Slot 4 is full.
2. Look for the next empty slot in the table.
3. **Linear Probing:** Check slot i+1, i+2, i+3 ... until you find an empty space.

Example:
- A wants slot 4 → full
- Check slot 5 → empty
- Park at slot 5

All inside the array!

Mechanical Analogy



Key Takeaways

- Open Addressing = No linked lists.
- Everything stays in the main array.
- Uses probing (e.g. linear probing).
- Fast and memory efficient, but may take someone else's spot (if the next slots are also full).

Same goal... different approach!

Comparison

Chaining (Open Hashing)	Open Addressing (Closed Hashing)
• Uses linked lists	• No linked lists
• Multiple items per slot	• All in main array
• More memory (pointers)	• Uses probing



Chain in the slot



Find the next empty slot

Two ways to handle collisions. **Same goal**: Find a place to store the data!

Different paths. Same destination!